

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.

A. Incorrect field:

- **Content-Length: -150**

Reason:

- **Content-Length** cannot be a negative value.
- It must be a **non-negative integer** representing the number of bytes in the response body.

2. Analyze the issue with `Content-Length` and explain its impact.

A. Issue:

- The value `-150` is invalid.

Impact:

- The browser cannot determine the correct size of the response body.
- The response may fail to load correctly.
- Some browsers may ignore the response or terminate the connection.
- It can lead to incomplete or corrupted page rendering.

3. Determine whether the status code matches the response correctness.

A. Status Code: `200 OK`

Meaning: The request was processed successfully.

Analysis:

Although `200 OK` indicates success, the response contains an invalid `Content-Length` header.

Therefore, the status code does not accurately reflect a completely correct response.

If the server cannot generate a valid response, it should return an appropriate error status (such as `500 Internal Server Error`) instead.

4. Suggest corrections for the identified errors.

A.

Replace `Content-Length: -150` with the actual response size (e.g., `150`).

Ensure `Content-Length` matches the exact number of bytes in the response body.

Keep the `200 OK` status only if the response is valid.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.

A.Errors:

- The `<p>` tag is not closed (`</p>` is missing).
- The `<!DOCTYPE html>` declaration is missing.
- It is good practice to include the `lang` attribute in the `<html>` tag.

2. Analyze how the browser will interpret this structure.

A. Modern browsers automatically correct minor HTML errors.

The browser will automatically insert the missing `</p>` tag before `</div>`.

Different browsers may handle invalid HTML slightly differently, leading to inconsistent behavior.

3. Determine the impact on rendering and layout.

A.

1. The page may still display **"Hello World"** correctly because browsers automatically fix the missing tag.
2. Invalid HTML can cause inconsistent layouts across different browsers.
3. CSS styles and JavaScript DOM manipulation may not work as expected due to the incorrect DOM structure.
4. It reduces code maintainability and standards compliance.

4. Suggest corrections to make the DOM valid.

A.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Test Page</title>
</head>
<body>

<div>
  <p>Hello World</p>
</div>

</body>
</html>
```

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL	Example Data	Visibility	Security Level
GET				

/login?user=admin&pass=1234 Visible in URL Low POST /login
Hidden in body Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.

A. GET

- Sends data as **URL parameters (query string)**.
- Data is visible in the browser's address bar.
- Can be bookmarked and stored in browser history.
- Suitable for retrieving data.

POST

- Sends data in the **HTTP request body**.
- Data is not visible in the URL.
- Cannot be bookmarked through the URL.
- Suitable for sending sensitive information such as login credentials

2. Analyze which method is more suitable for login and why.

A. **POST** is more suitable for login because:

- User credentials are sent in the request body instead of the URL.
- Passwords are not exposed in the browser history or address bar.
- It provides better security than GET.
- It is the standard method for authentication forms.

3. Identify security risks in using GET for authentication.

A. **POST** is more suitable for login because:

- User credentials are sent in the request body instead of the URL.
- Passwords are not exposed in the browser history or address bar.
- It provides better security than GET.
- It is the standard method for authentication forms.

4. Suggest best practices for secure login implementation.

A.

1. Use the **POST** method for login forms.
2. Use **HTTPS** to encrypt data during transmission.
3. Store passwords securely using hashing algorithms (e.g., bcrypt).
4. Validate input on both client and server.
5. Use secure session management and authentication tokens.
6. Enable CSRF protection and use strong password policies.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.

A.Missing closing tags:

- `</h1>` is missing.
- `</p>` is missing.

Note: The `<img>` tag is a **void (empty) element**, so it **does not require a closing tag**.

2. Analyze issues with the `<img>` tag.

A.Issues:

- The `<img>` tag does not have an `alt` attribute.
- The `alt` attribute is important for accessibility and is displayed if the image cannot be loaded.

Better usage:

```

```

3. Determine how browsers will handle these errors.

A.

1. Modern browsers automatically close the missing `<h1>` and `<p>` tags.
2. The page may still display correctly.
3. Different browsers may interpret the invalid HTML differently, causing layout inconsistencies.
4. Missing `alt` text reduces accessibility for screen readers and users if the image fails to load.

4. Suggest a corrected version of the code.

A.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
```

```
<h1>Welcome</h1>

<p>This is a paragraph.</p>



</body>
</html>
```

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.

A. Step 1: User enters a URL

- The user types a website address (e.g., www.example.com) into the browser.

Step 2: DNS Lookup

- The browser contacts the DNS server to convert the domain name into an IP address.

Step 3: HTTP Request

- The browser sends an HTTP request (GET or POST) to the web server.

Step 4: Server Processing

- The server processes the request, retrieves data if needed, and prepares a response.

Step 5: HTTP Response

- The server sends an HTTP response containing the requested web page or data.

Step 6: Browser Rendering

- The browser interprets the HTML, CSS, and JavaScript and displays the web page to the user.

2. Identify where delays can occur.

A.

1. **DNS lookup** may take time if the domain is not cached.
2. **Network latency** can delay the HTTP request and response.
3. **Server processing** may be slow due to heavy traffic or database queries.
4. **Large files** (images, videos, scripts) can increase page loading time.
5. **Browser rendering** may be slower for complex web pages.

3. Analyze the role of DNS in the process.

A.

DNS (**Domain Name System**) translates a domain name (e.g., www.example.com) into an IP address.

The browser uses this IP address to locate and connect to the correct web server.

Without DNS, users would need to remember numerical IP addresses instead of domain names.

4. Suggest optimizations to improve performance.

A.

1. Use **DNS caching** to reduce lookup time.
2. Enable **browser caching** for frequently accessed resources.
3. Compress files using **Gzip/Brotli**.
4. Minify HTML, CSS, and JavaScript files.
5. Optimize images and use modern formats (e.g., WebP).
6. Use a **Content Delivery Network (CDN)** to serve content from nearby servers.
7. Reduce unnecessary HTTP requests.
8. Optimize server and database performance.

Code of Conduct:

*I **Abburi Ganesh Ram Kamal** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some	Basic analysis with weak reasoning	No meaningful analysis provided

			justification		
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand